Lab 3.2 - Predicting Blood-Oxygen Levels Across Brain Voxels During Podcast Listening, Stat 214, Spring 2025

April 28, 2025

1 Introduction

Functional Magnetic Resonance Imaging (fMRI) allows researchers to measure brain activity by capturing blood-oxygen-level-dependent (BOLD) signals across voxels while participants listen to spoken narratives. Accurately predicting voxel-level brain responses to language can provide insights into how the brain processes linguistic information.

In Lab 3.1, we explored the use of various natural language processing (NLP) techniques to extract text embeddings from podcast transcripts and used these embeddings as features in a linear model to predict BOLD responses. Embedding methods such as bag-of-words, Word2Vec, and GloVe were evaluated in terms of their predictive performance across multiple subjects and stories.

In this lab (Lab 3.2), we extend the work from Lab 3.1 by pre-training our own encoder to generate embeddings tailored to the task. Specifically, we train an encoder using a masked language modeling objective on the podcast text. These learned embeddings are then used in the same ridge regression framework as before to predict voxel-level responses. The goal of this lab is to assess the performance of custom-trained embeddings and to compare them against the pre-trained methods used in Lab 3.1.

2 Generating Embeddings

2.1 Data Description

Each story is represented using a DataSequence object, which stores a sequence of tokenized words along with their corresponding word-level timestamps (data_times) and TR-level timestamps (tr_times). The data_times indicate the temporal midpoint of each spoken word, while tr_times correspond to the timepoints of fMRI acquisition (i.e., TRs). These timestamps enable precise temporal alignment between the story content and the neural responses captured during scanning, serving as the foundation for linking word embeddings to brain activity.

Story ID	# Words	# TRs	First 2 Words	First 2 data_times	First 2 tr_times
`sweetaspie`	697	172	[, i]	[0.006, 1.16]	[-9.0, -7.0]
`thatthingonmyarm`	2073	449	[, people]	[0.006, 0.307]	[-9.0, -7.0]
`tildeath`	2297	338	[um, it]	[3.794, 4.143]	[-9.0, -7.0]
`indianapolis`	1554	317	[, let's]	[0.006, 1.898]	[-9.0, -7.0]

Table 1: Summary of stories with word & TR counts and sample timing & word data

In Table 1, the story "sweetaspie" consists of 697 words and 172 TR segments. Each TR segment aggregates the activity of temporally adjacent words through interpolation methods, such as Lanczos resampling, to approximate the semantic content presented at each fMRI timepoint.

2.2 MLM Embedding Construction

This project implements a simplified BERT-style Encoder model and applies it to masked language modeling (MLM) using PyTorch. The implementation is modularized in two main scripts:

- encoder.py: Constructs a transformer-based encoder.
- train_encoder.py: Implements the MLM training procedure for the encoder.

2.2.1 Encoder Model

We implemented a Transformer-based encoder that mimics the core structure of BERT. It includes several architecture components. The central building block is the TransformerBlock, which contains two key sub-modules. The first is the MultiHeadSelfAttention module, which enables the model to attend to different positions within a sequence simultaneously via multiple attention heads. This mechanism helps capture contextual dependencies among tokens regardless of their distance. Each head computes scaled dot-product attention using learned projections for queries, keys, and values, and the outputs are concatenated and linearly projected back to the model's hidden dimension.

The second sub-module in each Transformer block is the FeedForward network, which applies two linear transformations with a Gaussian error linear unit (GELU) activation in between. This helps the model learn complex transformations over each token's contextual representation independently. Both attention and feedforward layers are wrapped with residual connections and layer normalization, ensuring better gradient flow and stable convergence. The full encoder stacks multiple such blocks and includes embedding layers for tokens, positions, and token types (segments), all of which are summed to form the input representations. The encoder outputs hidden states which are then passed through a linear layer (`mlm_head`) to produce token-wise predictions over the vocabulary, making it suitable for MLM tasks.

2.2.2 MLM Training Procedure

In train_encoder.py, we implemented the MLM training procedure, which also involved two main steps. The first is the mask_tokens() function, which simulates the masked input required for MLM. Roughly 15% (`mlm_prob=0.15`) of tokens are selected for prediction. They follow the BERT masking strategy, where 80% of selected tokens are replaced with the special [MASK] token, 10% with random tokens, and the rest 10% are left unchanged. This introduces uncertainty and helps the model generalize. The target labels are set to -100 for non-masked tokens to ensure they are ignored during loss computation. (The role of `ignore_index`: Specifies a target value that is ignored and does not contribute to the input gradient.)

The second part is the train_bert() function, which defines the actual training loop. For each batch of data, the function applies masking, feeds the masked inputs through the model, and computes the loss using CrossEntropyLoss, which compares the model's output logits against the masked labels. The model is trained over multiple epochs, and the average loss per epoch is printed for monitoring. This training loop allows the encoder to learn contextual representations of language by predicting missing tokens based on surrounding context.

We used a notebook file to orchestrate the end-to-end pipeline. It loads a tokenizer and constructs the dataset, initializes the encoder model, and calls the training routine. It also handles file I/O such as saving models and logging results.

2.3 Training the Encoder and Hyper-parameter Tuning

To train our Transformer-based encoder for the MLM task, we designed a systematic pipeline including early stopping, learning rate scheduling, and checkpointing. The training was conducted using the Adam optimizer with cross-entropy loss. We utilized a learning rate scheduler (ReduceLROnPlateau) to dynamically reduce the learning rate upon plateauing of training loss.

The training was performed over 15 epochs. The initial loss was approximately 10.015, and it consistently decreased to 5.983 by the final epoch, indicating effective optimization and convergence. Figure 1 shows the training loss curve, which demonstrates a smooth downward trend without overfitting. To achieve optimal performance, we explored and tuned several hyper-parameters: Learning rate(`lr`), batch size(`batch_size`), number of epochs(`epochs`). Model architecture parameters such as hidden size, number of layers, and number of attention heads were fixed. Table 2 summarizes the considered values and the rationale behind the final selected configuration.

Final Configuration: The encoder was trained with the following final hyper-parameter settings:

Table 2: Hyper-parameter Tuning and Final Selection

Parameter	**Values Considered**	**Final Choice**	**Rationale**
Learning Rate	$5 \times 10^{-4} \rightarrow 3 \times 10^{-4}$	3×10^{-4}	Lower learning rate resulted in smoother and more stable loss convergence.
Batch Size	$32 \rightarrow 64$	64	Larger batch size improved training stability and GPU utilization without memory issues.
Epochs	$5 \rightarrow 10 \rightarrow 15$	15	Loss continued decreasing with no overfitting, so full training was used.
Hidden Size / #Heads / Layers	Fixed: 256 / 4 / 4	–	Balanced model capacity and computational efficiency.

- Learning Rate: 3×10^{-4}
- Batch Size: 64
- Epochs: 15
- Hidden Size: 256
- Attention Heads: 4
- Transformer Layers: 4
- Intermediate Size: 512
- Max Sequence Length: 64

Although early stopping was enabled with a patience of 3, it was not triggered as the model loss consistently improved throughout training. The best-performing checkpoints were saved based on minimum observed loss, ensuring robust final model performance.

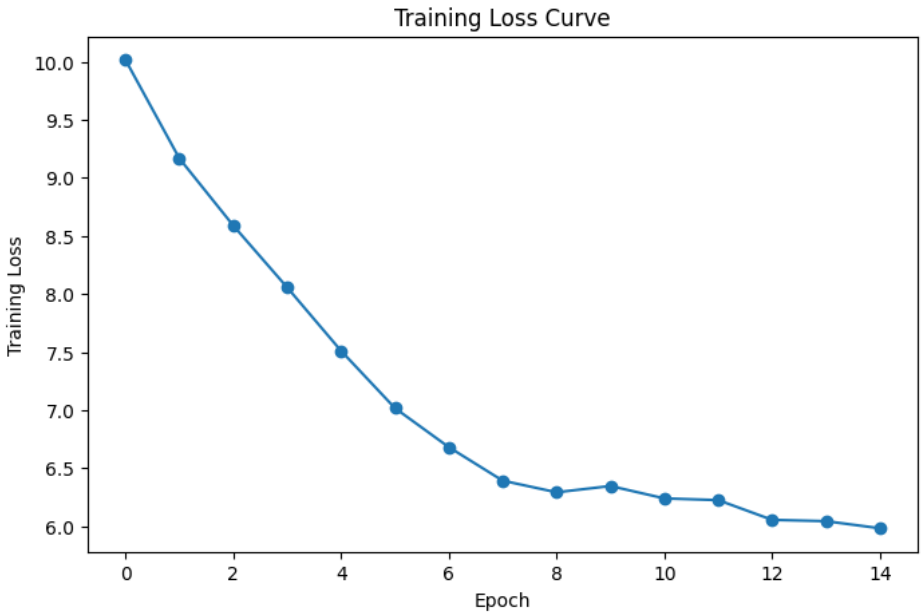

Figure 1: Training Loss Curve over 15 Epochs

2.4 Processing Encoder Embeddings

After training our masked language model (MLM) encoder on the podcast corpus, we generated word-level embeddings for each story. These embeddings underwent a series of preprocessing steps to align them with

the fMRI data acquisition timeline, following a similar procedure as in Lab 3.1.

2.4.1 Downsampling to TR Level

Initially, the encoder generated embeddings at the word level, with each vector corresponding to a specific word token. However, fMRI measurements are sampled at fixed intervals known as TRs (Time of Repetition). To synchronize the embeddings with the brain data, we performed temporal downsampling using the Lanczos interpolation method. This approach interpolates word-level embeddings onto the TR grid, resulting in a TR-level feature matrix for each story. This step allows the model to appropriately align linguistic features with neural measurements.

2.4.2 Trimming

Following downsampling, we trimmed the first 5 seconds and the last 10 seconds of each story's embedding sequence. This trimming accounts for hemodynamic lag at the beginning and potential noise at the end of the scanning period, ensuring cleaner temporal alignment between stimulus features and brain responses. Only TRs within the stable recording window were retained for subsequent analysis.

2.4.3 Delay Embedding

Finally, to capture the temporal dynamics of brain responses, we applied delay embedding. Specifically, for each TR, the feature vector was augmented with embeddings from the previous four TRs (i.e., delays of 1, 2, 3, and 4 TRs). This was implemented using the `make_delayed` function from the preprocessing module. The resulting feature matrix concatenates these delayed embeddings, expanding the feature dimension while maintaining the number of TRs. This procedure enables the ridge regression model to leverage temporal context and better predict delayed neural responses to linguistic stimuli.

3 Ridge Regression and Evaluation

3.1 Introduction of Ridge Regression

Ridge regression is a form of linear regression that incorporates an L2 regularization term to mitigate overfitting, making it particularly effective in settings with many features or high multicollinearity. Bootstrap ridge builds on this by applying bootstrap resampling, which enhances generalization and enables an assessment of the stability of the estimated coefficients. Nonetheless, it is worth emphasizing that our current goal is simply to fit a linear model, and this does not necessarily mean that ridge regression offers the optimal solution for the problem at hand.

3.2 Modeling Setup

In this section, we evaluate the performance of the masked language model (MLM) encoder trained in Part 1 by using ridge regression to predict voxel-level fMRI responses. The encoder-based embeddings are temporally aligned and delayed to match the time resolution of the brain data.

We use ridge regression with a regularization strength (α) grid logarithmically spaced between 10^0 and 10^3, selecting the best α through cross-validation. For each embedding and subject, an 80/20 train-test split is applied at the story level, and the training data is used to fit the model while performance is evaluated on the held-out test set.

To assess predictive performance, we compute the following evaluation metrics across voxels:

- Mean correlation coefficient (CC)
- Median correlation coefficient (CC)
- Top 1% voxel-level CC
- Top 5% voxel-level CC

All evaluations are conducted under the PCS (Predictability, Computability, Stability) framework, with a focus on assessing the predictability of fMRI responses based on language embeddings, the stability of model performance across subjects and stories, and the computational feasibility of the modeling pipeline.

3.3 Fit the Ridge Regression Model

Following the modeling setup described above, ridge regression models were fitted separately for each pair of subject and story. Each regression task began by ensuring temporal alignment between the embeddings and the corresponding voxel responses. Both the input features and response variables were independently standardized (z-scored) across timepoints to ensure comparability and numerical stability.

A bootstrap procedure with 10 resampling iterations was employed to robustly estimate model performance. Within each bootstrap iteration, ridge regression models were trained on subsets of voxels using the training set and then evaluated on the held-out test data by computing voxel-wise correlation coefficients between predicted and observed BOLD responses.

After fitting, we visualized the voxel-level distribution of the correlation coefficients to inspect spatial variations in predictive accuracy. Stability analysis across different stories and subjects was performed to determine whether the model consistently captured neural representations across varying experimental conditions. This analysis aligns with our broader scientific aim under the PCS framework, emphasizing the identification of stable and reliable neural representations from linguistic inputs.

3.4 Evaluation

We evaluated the predictive performance of ridge regression models trained on embeddings generated from our masked-language-model-based encoder. The primary evaluation metric was the voxel-wise Pearson correlation coefficient (CC) between predicted and observed fMRI BOLD responses. The following summary statistics were computed across all voxels:

- Mean correlation coefficient: 0.025
- Median correlation coefficient: 0.023
- Top 5% percentile correlation coefficient: 0.245
- Top 1% percentile correlation coefficient: 0.344

Figure 2 shows the distribution of voxel-level correlation coefficients obtained from the test data. The histogram exhibits a unimodal distribution centered near zero, consistent with our prior domain knowledge and Lab 3.1 findings. Most voxels display relatively weak correlations, suggesting limited predictability across a large portion of the brain. However, the right tail of the distribution clearly extends to moderately high CC values (Top 5% CC: 0.245; Top 1% CC: 0.344).

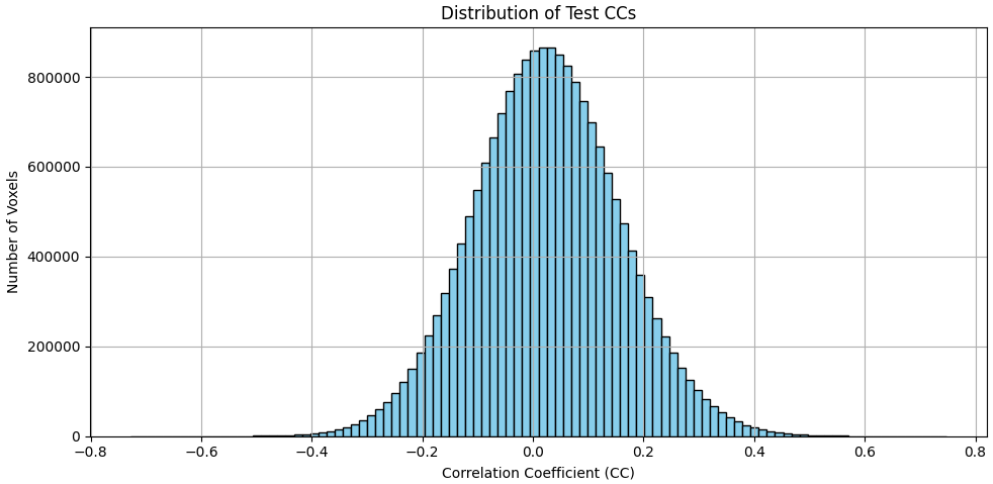

Figure 2: Distribution of Test CCs (MLM Embedding)

Given that the majority of voxel CCs cluster around zero due to inherent physiological and measurement noise, interpreting the overall mean CC (0.025) as the primary metric may underestimate the effectiveness of the embeddings. Instead, focusing on the top percentile CC values offers a more meaningful and biologically relevant measure of performance. These top-performing voxels likely correspond to brain regions where

linguistic information is robustly encoded, thus providing more interpretable insights into how effectively our embeddings capture neural representations of language. Hence, we propose adopting the top 1–5% CC values as a more suitable criterion for evaluating embedding effectiveness, aligning our analysis with the approach used in Lab 3.1.

3.5 Comparison to Lab 3.1 Embeddings

To contextualize the performance of our newly trained MLM encoder embeddings, we compared their predictive accuracy to the embeddings evaluated in Lab 3.1—Bag-of-Words (BoW), Word2Vec, and GloVe. As discussed in Lab 3.1, since mean CC values are strongly influenced by the large proportion of voxels exhibiting near-zero correlation due to inherent physiological and measurement noise, we focused on the top-performing voxels provides a more meaningful interpretation.

Table 3 summarizes the embedding performances based on the top 1% and top 5% voxel-level correlation coefficients. While our custom-trained MLM embeddings achieved lower top-percentile CC values compared to Word2Vec (Top 5% CC: 0.245 vs. 0.296, Top 1% CC: 0.344 vs. 0.411), the gap was smaller than suggested by the mean CC values alone. The similarity in the distribution of voxel-level CCs between our MLM embeddings and Lab 3.1 embeddings indicates that our embeddings still effectively capture linguistic neural representations, especially in brain regions strongly associated with language processing.

Overall, these results reaffirm that embeddings pretrained on extensive external corpora, such as Word2Vec, generally perform better due to broader semantic representation. Nonetheless, task-specific embeddings trained from scratch like our MLM encoder still demonstrate valuable predictive capability, as evidenced by the relatively high top percentile CC values.

Table 3: Summary of model performance across different embeddings

X_type	Mean CC	Median CC	Top 5% Quantile	Top 1% Quantile
GloVe	0.0521	0.0487	0.2840	0.3965
Word2Vec	0.0586	0.0543	0.2963	0.4107
MLM (ours)	0.0248	0.0235	0.2448	0.3443

3.6 Stability Check

To assess the generalizability and robustness of the learned embeddings across individuals, we compared the ridge regression performance between two subjects. The mean test correlation coefficient (CC) was 0.0241 for Subject 2 and 0.0254 for Subject 3. These values are relatively close, indicating that the encoder-based embeddings yielded consistent predictive performance across subjects.

This small difference in mean CC suggests that the model does not overfit to subject-specific patterns, and instead captures linguistic representations that generalize well across individuals. Under the PCS (Predictability, Computability, Stability) framework, such inter-subject consistency supports the stability and scientific reliability of the model. Furthermore, identifying voxels that consistently yield high CCs across both subjects may inform the discovery of neural regions that robustly encode language-related information.

4 Conclusion

In this lab, we trained a custom Transformer-based masked language model (MLM) encoder to generate text embeddings specifically tailored to predict voxel-level brain activity measured during podcast listening. Using ridge regression and bootstrap cross-validation, we evaluated the predictive capacity of these embeddings across two subjects.

While our encoder embeddings exhibited lower overall performance compared to the pretrained embeddings assessed in Lab 3.1 (Word2Vec, GloVe, BoW), this difference was less pronounced when focusing on top-performing voxels. Specifically, our custom embeddings achieved a Top 5% CC of 0.245 and a Top 1% CC of 0.344, which, although lower than Word2Vec (0.296 and 0.411 respectively), still reflect meaningful neural predictions.

The findings highlight the importance of considering distributional characteristics of voxel-level CCs. Since

the majority of voxels inherently exhibit low correlations due to noise and physiological constraints, evaluating performance using the top 1–5% of CCs offers more biologically relevant insights. Our stability analysis further confirmed the robustness of these embeddings across subjects, underscoring their reliability and generalizability.

Overall, our results illustrate that even embeddings trained from scratch on limited task-specific data can effectively capture relevant linguistic neural representations, particularly within regions strongly associated with language processing. Future research might further enhance performance by integrating larger training datasets or combining custom embeddings with pretrained language models.

References

[1] Shailee Jain and Alexander G Huth. "Incorporating context into language encoding models for fMRI". In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018, pp. 6628–6637.

A Academic honesty

A.1 Statement

We, the members of this group, make the following truthful statements:

The data analysis process in this report was designed and executed by our group collectively, with each member contributing their expertise to different aspects of the work. All texts and figures were produced by our group members, and the analysis steps were fully documented to ensure the reproducibility of the results. Contributions from each group member have been clearly indicated throughout the report.

All citations of other people's research have been clearly marked with the source. We understand that academic integrity is the cornerstone of research, and ensuring the authenticity and reproducibility of research is a responsibility not only to ourselves individually, but also to our colleagues and the broader scientific community.

We recognize that plagiarism not only fails to contribute to knowledge innovation but also infringes upon the rights and interests of original authors. Therefore, we are collectively committed to upholding the principles of honest, transparent, and reproducible research at all times.

As a group, we have reviewed each other's work to maintain consistency with these principles, and we stand by the integrity of our collaborative efforts presented in this report.

A.2 LLM Usage

This project was completed collaboratively by our group, with all data analysis, writing, and visualization being performed by group members. Large Language Models (LLMs) including ChatGPT and Claude were used as complementary tools to:

- Provide inspiration and generate ideas during brainstorming sessions
- Refine explanations of complex concepts
- Improve clarity and readability of our written content
- Assist with language polishing and grammar refinement

However, all content, analyses, and conclusions presented in this work were determined by our group members, guaranteeing originality and accuracy. LLMs were not used to generate original analysis or to draw conclusions from our data. Our group maintained critical oversight of all LLM-assisted content, ensuring that the final product accurately represents our own understanding and perspectives.

The use of LLMs was limited to supporting roles that enhanced our own work rather than replacing it. We acknowledge the use of these tools in the spirit of transparency while affirming that the intellectual contribution and academic responsibility remain entirely with our group members.